SSDT hooking alternative in x64 systems

Asked 12 years, 11 months ago Modified 3 years ago Viewed 5k times

4

I read a little bit and I find out that SSDT hooks using drivers in Windows 7 x64 systems are harder, on purpose because of Patch Guard/Driver Signing while in x32 systems that doesn't happen.

So, is there any other alternative for x64 systems? I mean, is there any other way that I could go to achieve the same result? (global hook a ntdll api)

windows 64-bit kernel hook low-level

Share Improve this question Follow

edited Nov 12, 2012 at 16:36
Keagan Ladds
458 ● 3 ● 15

asked Jul 22, 2012 at 5:06
Leandro Battochio
364 ● 6 ● 22

3 The whole *point* of disabling SSDT in x64 was so that people couldn't install global hooks. – Harry Johnston Jul 23, 2012 at 4:20

so @Harry, you're saying it's impossible to install global hooks in x64 systems? I mean, without forcing the user to disable PG. – Leandro Battochio Jul 23, 2012 at 19:07

1 Technically, no, it's not impossible - malware writers have figured out various ways of doing this. But going down that path would be a very bad idea: en.wikipedia.org/wiki/Sony_rootkit – Harry Johnston Jul 23, 2012 at 21:49

@HarryJohnston can you share more about this subject? ways of doing it for example? – Leandro Battochio Jul 24, 2012 at 1:53

3 Answers

Sorted by: Highest score (default)

You could implement User Mode hooks using the DLL Injection method as this works on both x86 and x64. If you want to make the hook global you will need to inject the DLL into every process including newly created ones.

1

Share Improve this answer Follow

answered Nov 12, 2012 at 16:28

Keagan Ladds
458 3 15

0

SSDT is not allowed in x64. File system or mini-filter driver is a way to alter any default system behavior. What do you want to achieve?

Share Improve this answer Follow

answered Jul 22, 2012 at 5:41

Rohan
53.4k 12 95 89

I want to hook NtOpenProcess in a level that it "works" for every application running in my PC. – Leandro Battochio Jul 22, 2012 at 5:50

Disable PG and use SSDT hook. :) – Xearinox Jul 22, 2012 at 7:19

Not very good for normal users. I didn't want to force them to install any other stuffs – Leandro Battochio Jul 22, 2012 at 17:03

3 If your target audience understand the issues involved with installing global hooks, and assuming your application is valuable enough to them, they won't mind disabling PG in order to use it. If they *don't* understand the issues, then they aren't able to give informed consent, and it would be unethical of you to install a global hook on their system even if you found a way to do so. – Harry Johnston Jul 23, 2012 at 4:23

It is an anti-cheater program, they arent supposed to know how it works, but I cant force them to disable PG, since it is for "advanced" users and the audience of my exe is normal users, you know? Probably most of them wouldnt know how to disable it. – Leandro Battochio Jul 23, 2012 at 5:50

0

It's up to what your target is. If you need to prevent malicious operation on process or thread, you can Use NotifyRoutine or ObRegisterCallbacks instead. Or you are able to use minifilter to monitor file operation.

Windows SSDT Hook（二）

原创 摩尔の 于 2025-05-30 12:17:39 发布 阅读量1k 收藏 30 点赞数 14 CC 4.0 BY-SA版权
分类专栏： 驱动开发 Hook 文章标签： windows内核

获取SSDT基址

在32位Windows系统中，SSDT（System Service Descriptor Table）作为系统服务调用的核心数据结构，其基址信息是公开且稳定的。由于`ntoskrnl.exe`内核模块默认导出`KeServiceDescriptorTable`符号，开发者只需使用`extern`声明即可直接访问：

c AI写代码 复制 运行

```c
extern PKSERVICE_TABLE_DESCRIPTOR KeServiceDescriptorTable;
```

然而，64位系统架构带来了重大变化。微软出于安全考虑，不再导出SSDT相关符号，并引入了以下保护机制：

1. SSDT表基址变为动态变化，每次系统启动都会重新随机化
2. 移除了直接的符号导出
3. 增加了内核`PatchGuard`保护
   因此，想要获取64位SSDT基址就需要一些特殊技术手段来实现，常见方法包括：

- 借助第三方符号（PDB）或调试器

```
地址(A): nt!KiSystemCall64
查看选项
  显示代码字节  显示地址
  显示源代码    显示符号名
  显示行号
fffff801`0862bd7c 83e720            and edi,20h
fffff801`0862bd7f 25ff0f0000        and eax,0FFFh
nt!KiSystemServiceRepeat:
fffff801`0862bd84 4c8d1535fb9e00    lea r10,[nt!KeServiceDescriptorTable (fffff801`0901b8c0)]
fffff801`0862bd8b 4c8d1daeac8e00    lea r11,[nt!KeServiceDescriptorTableShadow (fffff801`08f16a40)]
fffff801`0862bd92 f7437880000000    test dword ptr [rbx+78h],80h
fffff801`0862bd99 7413              je nt!KiSystemServiceRepeat+0x2a (fffff801`0862bdae)
fffff801`0862bd9b f7437800002000    test dword ptr [rbx+78h],200000h
fffff801`0862bda2 7407              je nt!KiSystemServiceRepeat+0x27 (fffff801`0862bdab)
ff  0xFFFFF801  0xfffff801  3d1d55ae8e00   lea r11,[nt!KeServiceDescriptorTableFilter (fffff801`08f16c00)]
fffff801`0862bdab 4d8bd3            mov r10,r11
fffff801`0862bdae 413b443a10        cmp eax,dword ptr [r10+rdi+10h]
fffff801`0862bdb3 0f83f6070000      jae nt!KiSystemServiceExitPico+0x2c3 (fffff801`0862c5af)
fffff801`0862bdb9 4d8b143a          mov r10,qword ptr [r10+rdi]
fffff801`0862bdbd 4d631c82          movsxd r11,dword ptr [r10+rax*4]
145 %

Debugger Immediate Window
0: kd> db KeServiceDescriptorTable
fffff801`0901b8c0  b0 1a 2e 08 01 f8 ff ff-00 00 00 00 00 00 00 00  ................
fffff801`0901b8d0  d9 01 00 00 00 00 00 00-18 22 2e 08 01 f8 ff ff  .........".......
```

CSDN @摩尔の

- 通过 `msr` 寄存器读取

通过读取 `msr 0xC0000082` 可得到 `KiSystemCall64` 的函数地址，从此函数地址向下可以定位到 `KiSystemServiceRepeat` 函数，从而间接获取 SSDT。

```
.text:0000000140070FDE
.text:0000000140070FDE                          KiSystemServiceStart:                   ; DATA XREF: KiServiceInternal+5A↑o
.text:0000000140070FDE                                                                  ; .data:00000001401E6618↓o
.text:0000000140070FDE 48 89 A3 D8 01 00 00                     mov     [rbx+1D8h], rsp
.text:0000000140070FE5 8B F8                                    mov     edi, eax
.text:0000000140070FE7 C1 EF 07                                 shr     edi, 7
.text:0000000140070FEA 83 E7 20                                 and     edi, 20h
.text:0000000140070FED 25 FF 0F 00 00                           and     eax, 0FFFh
.text:0000000140070FF2
.text:0000000140070FF2                          KiSystemServiceRepeat:                  ; CODE XREF: KiSystemCall64+47B↓j
.text:0000000140070FF2 4C 8D 15 47 78 23 00                     lea     r10, KeServiceDescriptorTable
.text:0000000140070FF9 4C 8D 1D 80 78 23 00                     lea     r11, KeServiceDescriptorTableShadow
.text:0000000140071000 F7 83 00 01 00 00 80 00                  test    dword ptr [rbx+100h], 80h
.text:0000000140071000 00 00
```

CSDN @摩尔の

从上图中可以看出，`4C 8D 15[lea r10]` 指令后面的操作数即为SSDT地址的相对偏移，因此在通过特征值定位到此处时，可以通过以下公式计算出实际的SSDT地址。

> SSDT地址 = Rip + offset = 0x140070FF9 + 0x237847

这种方法的实现在网络上可以轻松获取，这里就不详细展开了。值得注意的一点是，win10 1903版本之后，由于引入了内核隔离机制，通过`msr`获取到的地址不再是`KiSystemCall64`，而是`KiSystemCall64Shadow`，这也导致了无法按照以上方法找到SSDT表。

```
0: kd> rdmsr 0xc0000082
msr[c0000082] = fffff801`08c311c0
0: kd> u fffff801`08c311c0
nt!KiSystemCall64Shadow:
fffff801`08c311c0 0f01f8                  swapgs
fffff801`08c311c3 654889242510900000      mov     qword ptr gs:[9010h],rsp
fffff801`08c311cc 65488b242500900000      mov     rsp,qword ptr gs:[9000h]
fffff801`08c311d5 650fba24251890000001    bt      dword ptr gs:[9018h],1
fffff801`08c311df 7203                    jb      nt!KiSystemCall64Shadow+0x24 (fffff801`08c311e4)
fffff801`08c311e1 0f22dc                  mov     cr3,rsp
fffff801`08c311e4 65488b242508900000      mov     rsp,qword ptr gs:[9008h]
fffff801`08c311ed 6a2b                    push    2Bh
0: kd> u KiSystemCall64
nt!KiSystemCall64:
fffff801`0862ba00 0f01f8                  swapgs
fffff801`0862ba03 654889242510000000      mov     qword ptr gs:[10h],rsp
fffff801`0862ba0c 65488b2425a8010000      mov     rsp,qword ptr gs:[1A8h]
fffff801`0862ba15 6a2b                    push    2Bh
fffff801`0862ba17 65ff342510000000        push    qword ptr gs:[10h]
fffff801`0862ba1f 4153                    push    r11
fffff801`0862ba21 6a33                    push    33h
fffff801`0862ba23 51                      push    rcx
0: kd> u KiSystemServiceRepeat
nt!KiSystemServiceRepeat:
fffff801`0862bd84 4c8d1535fb9e00          lea     r10,[nt!KeServiceDescriptorTable (fffff801`0901b8c0)]
fffff801`0862bd8b 4c8d1daeac8e00          lea     r11,[nt!KeServiceDescriptorTableShadow (fffff801`08f16a40)]
```

CSDN @摩尔の

- 通过特征码扫描内核映像

由于无法再通过 `msr` 来实现查找，因此在此基础上又延伸出了类似的方案。即通过某些确定的系统函数地址，向上或向下寻找SSDT的特征值。通过IDA分析内核模块中 `Zw*` 系统调用接口（这里并非ntdll中的用户态调用接口，而是提供给wdm驱动使用的内核态接口），可以看出内核模块中的系统调用实际上同样是通过SSDT表实现的。以 `ZwCreateFile` 函数为例，内部调用 `KiServiceInternal` ，然后通过 `jmp` 指令跳转到 `KiSystemCall64` 函数中。

```
.text:000000014006AF00
.text:000000014006AF00 48 8B C4                          mov     rax, rsp
.text:000000014006AF03 FA                                cli
.text:000000014006AF04 48 83 EC 10                       sub     rsp, 10h
.text:000000014006AF08 50                                push    rax
.text:000000014006AF09 9C                                pushfq
.text:000000014006AF0A 6A 10                             push    10h
.text:000000014006AF0C 48 8D 05 DD 27 00 00              lea     rax, KiServiceLinkage
.text:000000014006AF13 50                                push    rax
.text:000000014006AF14 B8 52 00 00 00                    mov     eax, 52h ; 'R'
.text:000000014006AF19 E9 22 5F 00 00                    jmp     KiServiceInternal
.text:000000014006AF19                       ZwCreateFile    endp
```

CSDN @摩尔の

```
.text:0000000140070E40                       var_18          = qword ptr -18h
.text:0000000140070E40
.text:0000000140070E40 48 83 EC 08                           sub     rsp, 8
.text:0000000140070E44 55                                    push    rbp
.text:0000000140070E45 48 81 EC 58 01 00 00                  sub     rsp, 158h
.text:0000000140070E4C 48 8D AC 24 80 00 00 00               lea     rbp, [rsp+80h]
.text:0000000140070E54 48 89 9D C0 00 00 00                  mov     [rbp+0E8h+var_28], rbx
.text:0000000140070E5B 48 89 BD C8 00 00 00                  mov     [rbp+0E8h+var_20], rdi
.text:0000000140070E62 48 89 B5 D0 00 00 00                  mov     [rbp+0E8h+var_18], rsi
.text:0000000140070E69 FB                                    sti
.text:0000000140070E6A 65 48 8B 1C 25 88 01 00               mov     rbx, gs:188h
.text:0000000140070E6A 00
.text:0000000140070E73 0F 0D 8B D8 01 00 00                  prefetchw byte ptr [rbx+1D8h]
.text:0000000140070E7A 0F B6 BB F6 01 00 00                  movzx   edi, byte ptr [rbx+1F6h]
.text:0000000140070E81 40 88 7D A8                           mov     [rbp+0E8h+var_140], dil
.text:0000000140070E85 C6 83 F6 01 00 00 00                  mov     byte ptr [rbx+1F6h], 0
.text:0000000140070E8C 4C 8B 93 D8 01 00 00                  mov     r10, [rbx+1D8h]
.text:0000000140070E93 4C 89 95 B8 00 00 00                  mov     [rbp+0E8h+var_30], r10
.text:0000000140070E9A 4C 8D 1D 3D 01 00 00                  lea     r11, KiSystemServiceStart
.text:0000000140070EA1 41 FF E3                              jmp     r11
.text:0000000140070EA1                       KiServiceInternal endp
.text:0000000140070EA1
.text:0000000140070EA1                       ; ---------------------------------------------------------------------------
.text:0000000140070EA4                       align_140070EA4:                        ; DATA XREF: .pdata:000000014027E348↓o
.text:0000000140070EA4 66 66 66 66 66 66 66 0F…              align 20h
.text:0000000140070EC0
.text:0000000140070EC0                       ; =============== S U B R O U T I N E =======================================
.text:0000000140070EC0
.text:0000000140070EC0
.text:0000000140070EC0                       ; __int64 __fastcall KiSystemCall64(void *, __int64, __int64, __int64)
.text:0000000140070EC0                       KiSystemCall64  proc near               ; DATA XREF: .pdata:000000014027E354↓o
```

CSDN @摩尔の

c

AI写代码 复制 运行

```c
PLONG HookHandler::GetSSDTBase() {
#ifdef _WIN64
    const PVOID zw_close = ZwClose;

```

```
    const ULONG offset = 0x400;
    static PSSDTEntry ssdt_address = nullptr;
    __try {
        if (ssdt_address) return ssdt_address->ServiceTableBase;

        PUCHAR service_internal_address = nullptr;
```

展开

除了上述方法，在网上还见过一种通过在 `_strnicmp` 函数到 `KdDebuggerNotPresent` 之间搜索 `8b f8 c1 ef 07 83 e7 20 25 ff 0f 00 00` 的定位方式。

SSDT解析

前一篇中提到SSDT中的 `ServiceTableBase` 指向的一个 `ULONG` 数组，32位系统中的数组元素代表函数地址，而在64位系统中，由于ULONG类型无法存储完整的指针类型，因此实际存储的是函数地址相对于SSDT基址的"偏移"。需要注意的是这里的偏移并不仅仅是简单的偏移，而是需要固定公式计算出的偏移长度。

```
.text:0000000140070FF2
.text:0000000140070FF2                 KiSystemServiceRepeat:                  ; CODE XREF: KiSystemCall64+47B↓j
.text:0000000140070FF2 4C 8D 15 47 78 23 00            lea     r10, KeServiceDescriptorTable
.text:0000000140070FF9 4C 8D 1D 80 78 23 00            lea     r11, KeServiceDescriptorTableShadow
.text:0000000140071000 F7 83 00 01 00 00 80 00         test    dword ptr [rbx+100h], 80h
.text:0000000140071000 00 00
.text:000000014007100A 4D 0F 45 D3                     cmovnz  r10, r11
.text:000000014007100E 42 3B 44 17 10                  cmp     eax, [rdi+r10+10h]
.text:0000000140071013 0F 83 E9 02 00 00               jnb     loc_140071302
.text:0000000140071019 4E 8B 14 17                     mov     r10, [rdi+r10]
.text:000000014007101D 4D 63 1C 82                     movsxd  r11, dword ptr [r10+rax*4] ; offset = SSDT[sizeof(ULONG) * index]
.text:0000000140071021 49 8B C3                        mov     rax, r11
.text:0000000140071024 49 C1 FB 04                     sar     r11, 4          ; offset = offset >> 4
.text:0000000140071028 4D 03 D3                        add     r10, r11        ; addr = (LONGLONG)SSDT + (LONGLONG)offset
.text:000000014007102B 83 FF 20                        cmp     edi, 20h ;
.text:000000014007102E 75 50                           jnz     short loc_140071080
.text:0000000140071030 4C 8B 9B B8 00 00 00            mov     r11, [rbx+0B8h]
```

CSDN @摩尔の

c　AI写代码　复制　运行

```c
// 计算函数地址
PVOID func_addr = (PVOID)(((LONG)ssdt_base[info.func_index] >> 4) + (ULONGLONG)func_addr);
// 计算偏移
ssdt_base[info.func_index] = (LONG)((ULONGLONG)target_addr - (ULONGLONG)ssdt_base) << 4
```

跳转指令

由于 `ServiceTableBase` 中存储的实际上是相对于SSDT基址的"偏移"，因此hook函数的地址必须在以SSDT基址为中心的4GB内存范围内。而通常情况下我们自己的驱动的加载地址必然在4GB范围外，因此我们不能简单的将Hook函数的地址写入SSDT表，而应该在范围内选择一块地址作为函数跳转的中转。

，选择合适的跳转指令

们选择长度最小的12字节跳转法，实现从中转地址跳转到Hook函数。

c AI写代码 复制 运行

```c
#pragma pack(push, 1)
struct HookShellCode {
    USHORT mov;
    ULONG_PTR addr;
    UCHAR push;
    UCHAR ret;
};
#pragma pack(pop)
HookShellCode shell_code;
shell_code.mov = 0xB848;
```

展开

第二步，遍历所有的代码段，从中找出合适的中转地址

c AI写代码 复制 运行

```c
PULONG kernel_base = GetKernelBase();// 获取内核模块基址
if (!kernel_base) return nullptr;

// Dos Header
PIMAGE_DOS_HEADER dos_header = (PIMAGE_DOS_HEADER)kernel_base;
if (dos_header->e_magic != IMAGE_DOS_SIGNATURE) return nullptr;

// NT Header
PIMAGE_NT_HEADERS nt_header = (PIMAGE_NT_HEADERS)((PUCHAR)dos_header + dos_header->e_lfanew);
if (nt_header->Signature != IMAGE_NT_SIGNATURE) return nullptr;
```

展开

PatchGuard

基于以上内容，就可以实现在64位系统上的SSDT Hook了。然而，当实际运行Hook驱动程序时，会发现正常运行一段时间后，系统就会触发BSOD(错误代码：0x109，内核检测到关键内核代码或数据损坏)，而这就是`PatchGuard`的作用了。

到目前为止，通过`PatchGuard`的方法未作深入研究，仅通过网络得知可能存在以下两种方法：

- 通过某种方式伪装骗过PG检测，例如EPT Hook等。
- 直接与PG对抗